Precision Strike: Targeted Misclassification of Accelerated CNNs with a Single Clock Glitch

Arsalan Ali Malik[1,2], Furkan Aydin[2], and Aydin Aysu[1,2]
[1]Department of Electrical and Computer Engineering, North Carolina State University, Raleigh, USA
[2]MithrilAI Corp., Cary, USA
{aamalik3, aaysu}@ncsu.edu, furkan.mithrilai@gmail.com

***Abstract*—Fault injection attacks (FIAs) present a significant threat to the integrity of deep neural networks (DNNs), particularly in hardware-accelerated deployments on field-programmable gate arrays (FPGAs). These attacks intentionally introduce faults into the system, leading the DNN to generate incorrect outputs. This work presents the first successful targeted misclassification attack against a convolutional neural network (CNN) implemented on FPGA hardware, achieved by injecting a single clock glitch at the final layer (argmax) to manipulate the predicted output class. Our attack targets the commonly adopted argmax layer, a lightweight replacement for softmax in resource-constrained implementations. By precisely injecting a single clock glitch during the comparison phase of the argmax operation, the attack reliably induces misclassifications, forcing the model to 'skip' a specifically chosen class and output an incorrect label for it without affecting the computed scores of other classes.**

Unlike prior works that only cause random misclassifications, our attack achieves a high success rate of 80–87% for a targeted class, without inducing collateral misclassifications of other classes. Our evaluations show a significant reduction in classification accuracy, with the model's performance dropping from an initial 94.7% to an average final accuracy ranging from 7.7–14.7%. Our attack is demonstrated on a CNN model implemented using a common systolic array architecture, which is well-suited for resource-constrained edge devices and artificial intelligence (AI) accelerators. Our study confirms the vulnerability of hardware-accelerated machine learning systems to low-cost physical attacks, emphasizing the critical need for hardware-level countermeasures in safety-critical machine learning applications.

***Index Terms*—Fault Injection Attacks (FIA), Clock Glitch, Convolutional Neural Network (CNN), MNIST Dataset, Field-Programmable Gate Array (FPGA), Hardware Security**

I. Introduction

Deep neural networks (DNNs) have emerged as a central technology in modern artificial intelligence (AI), enabling high-performance solutions in image recognition, natural language processing, autonomous systems, and medical diagnostics [1–3]. Their ability to learn complex patterns from large datasets makes them ideal for deployment in a wide range of embedded and real-time applications. As DNNs are increasingly integrated into edge AI devices—such as autonomous drones, portable health monitors, and smart surveillance systems—there is growing interest in optimizing their inference performance under hardware constraints, such as memory, size, weight, and power.

To meet performance and power demands, DNN inference is increasingly offloaded to specialized hardware accelerators, particularly field-programmable gate arrays (FPGAs), in edge AI devices. These platforms offer flexibility and energy efficiency while supporting on-device inference without relying on cloud infrastructure and constant connectivity. However, their deployment in hardware introduces new security risks. In particular, hardware-level attacks can compromise model integrity by manipulating computations during inference or leaking sensitive information [4–10]. These attacks are especially concerning in resource-constrained environments, where lightweight post-processing components, such as the final classification logic, may lack robust protection mechanisms.

Among such threats are fault injection attacks (FIAs) that are a powerful class of physical attacks that intentionally perturb the hardware during execution, forcing incorrect or adversary-controlled outcomes [4, 11, 12]. One widely adopted FIA technique is clock glitching, where the attacker momentarily manipulates the clock signal, offering a combination of low cost, implementation simplicity, and high precision. Prior works have demonstrated how FIAs can degrade DNN accuracy or cause misclassifications. However, these attacks primarily (i) aim to randomly change the output class [5, 13], (ii) lack emphasis on more evasive attacks that selectively impact only one or a few classes, and (iii) target earlier layers or assume extensive fault propagation [9, 10]. As a result, evasive attacks, which affect only specific classes while leaving others unaffected, remain underexplored. Additionally, limited research focuses on how the output layer's lightweight decision logic, such as the argmax function used in final classification stages, behaves under precise physical faults.

In this paper, we propose an evasive clock glitch attack on a convolutional neural network (CNN) trained on the MNIST dataset, a widely used benchmark for handwritten digit classification [14]. By following conventional systolic array architectures used in commercial products, we implement CNN inference on FPGA hardware to reflect typical edge AI deployment settings, where efficiency and compact design are prioritized due to platform limitations. Our attack specifically targets the final stage of the inference process by injecting a *single* clock glitch during the argmax operation, which determines the predicted class label. We show that this decision-making logic—despite its simplicity and efficiency—is susceptible to precise physical fault injection, enabling *targeted* misclassifications by causing the system to 'skip' targeted class labels.

To the best of our knowledge, this is the *first* demonstration of a single-glitch attack on the argmax layer of an FPGA-based CNN. While prior research has focused on fault injection in CNNs, these studies have not explored vulnerabilities in

the argmax layer, particularly in the context of resource-constrained edge devices, FPGA accelerators, and *single-glitch* attacks [5, 9, 13, 15, 16].

The contributions of this work are as follows:

1) We present the *first single-clock glitch* attack targeting the sequential implementation of the argmax function in FPGA-based CNNs. The argmax function, which resides in the final layer, plays a critical role in determining the classification outcome. Our approach enables an adversary to deliberately manipulate the model's output by injecting a precisely timed fault into the final layer, effectively skipping specific targeted class labels and causing the model to misclassify.
2) We implement a CNN on FPGA hardware using a systolic array architecture with fixed-point quantization and a sequential argmax function to reflect edge device constraints. We validate this hardware-aware design through a full implementation on the MNIST dataset using the ChipWhisperer CW305–100T platform. The resulting architecture enables compact realization and is well-suited for deployment on low-cost embedded inference systems.
3) We demonstrate the effectiveness of our proposed attack by targeting the argmax function in the final layer of a CNN. When applied to an MNIST-trained model implemented on an FPGA platform, the attack achieved a success rate of up to 87%. By applying a carefully timed clock glitch during the execution of the sequential argmax, the attack reduces inference accuracy from an initial 94.7% to a final accuracy ranging from 7.7–14.7% on average, verifying the feasibility of physical attacks on classification logic in resource-constrained systems.

II. Preliminaries

A. DNNs

DNNs are a type of machine learning algorithm made up of multiple layers of interconnected nodes, or neurons [17]. CNNs are a prominent type of DNN, particularly effective for image processing tasks [18]. They typically comprise convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected (dense) layers for classification [19]. In classification tasks, these models typically conclude with a softmax or, in hardware-constrained deployments, an argmax layer to produce the final output label.

B. Argmax Layer

The argmax is a deterministic function frequently deployed at the final stage of classification models, such as CNNs. Formally, given a vector of unnormalized class logits $\mathbf{z} = [z_1, z_2, \ldots, z_n]$, where each z_i denotes the confidence score associated with class i, the argmax function is defined as:

$$argmax(\mathbf{z}) = \underset{i}{argmax}\ z_i.$$

The argmax deterministically outputs the index i corresponding to the largest value z_i in the input vector, thereby identifying the predicted class. It operates by comparing all class scores and selecting the maximum among them [20]. Argmax is typically used in the final layer of a neural network to determine the predicted class. As a non-learnable component, its behavior is highly sensitive to small variations in the input vector, particularly when class scores are closely spaced. Disruptions in the comparison sequence—whether through noise, faults, or adversarial interference—can cause it to select an incorrect index, resulting in misclassifications.

C. FIAs

FIAs are a form of active physical attack in which an adversary deliberately introduces faults or errors into a system's computation—typically through methods such as voltage glitches, clock manipulation, laser pulses, or electromagnetic interference—with the goal of altering its normal behavior [21]. These attacks target computational aspects such as memory, processing units, or timing mechanisms to disrupt the normal flow of data and introduce computational errors. FIAs are typically used in crypto systems to steal secret cryptographic keys and has been successfully employed against various systems, including smart cards, embedded platforms, and cryptographic modules [22, 23].

Among the various FIA methods, clock glitch attacks have gained significant traction due to their simplicity and effectiveness [4, 9, 24, 25]. In these attacks, the adversary manipulates the clock signal driving the hardware, causing temporary disruptions in the timing of computations. These glitches can lead to synchronization errors, incorrect data processing, or missed operations [4]. Clock glitch attacks are considered low-cost because they do not require any physical tampering with the hardware. A clock glitch is defined by two components: *glitch width* (the duration of the disruption), and *glitch offset* (the timing of the glitch relative to the standard clock cycle) [4, 9]. Oftentimes, a third parameter, *external offset*, is also used, which defines the delay from the moment the trigger is detected until the glitch is injected [15]. This lightweight attack requires minimal hardware modifications and only slight adjustments to the clock signal to cause fatal damage.

Several prior works have explored FIAs on DNNs deployed on edge devices, focusing on hardware vulnerabilities during critical operations [5, 13, 23, 26–28]. These studies often target layers such as softmax, sigmoid, and dense, using fault injection methods like bit-flip and stuck-at faults to manipulate the final predicted class, leading to misclassifications [29]. However, these attacks typically either (i) aim to cause random misclassifications [5], or (ii) are carried out at the software or simulation level [9], overlooking the physical aspects of fault injection that manipulate hardware during runtime. As such, they fall outside the scope of this work.

D. Class Skip Attack

This work introduces a class skip attack, where 'skipping' refers to a fault-induced disruption in the comparison phase of the argmax operation, causing the system to omit the correct class label (classification) and select an incorrect one, leading to misclassification. In a normal execution, the argmax function sequentially compares the output scores of

all candidate classes and selects the one with the highest value as the predicted label. However, when a clock glitch is injected during this comparison phase, it can disrupt or corrupt the register update process within the sequential logic. As a result, the correct class score may fail to be registered, causing the system to skip over it and instead select a lower-scoring class—ultimately leading to misclassification.

Unlike software-level instruction skip attacks [15, 30], which modify the control flow in microcontroller-based AI systems, this FIA is carried out at the hardware level. By precisely injecting a clock glitch during a critical clock cycle, the comparator's internal state is corrupted without altering any software instructions or execution paths. This hardware-centric manipulation directly affects the decision-making logic, making it fundamentally different from conventional software-based fault attack strategies.

III. Related Work

Lee et al. demonstrated the feasibility of clock glitch attacks on DNNs running on microcontrollers, targeting the softmax and sigmoid activation functions in the output layer [15]. They induce misclassification by either skipping the `for loop` statement (instruction skipping) or flipping the sign bit in sigmoid, focusing on control-flow manipulation rather than broader data path disruptions. Their attacks reduce accuracy to 16.09% for multi-class classification and 1.2% for binary classification. Shahin et al. reviewed the vulnerability of AI accelerators and AI IP cores to FIAs, emphasizing a shift from traditional targets like cryptographic devices [31]. Faiz et al. explore vulnerabilities in DNNs for security attacks, focusing on challenges in adversarial attack implementation and strategies to develop more robust DNN systems [32].

The work by Ordoñez et al. is most closely related to ours but has several limitations [13]. Their proposed methodology is explicitly implemented on a deep-learning processing unit (DPU) deployed on an AMD KRIA KV260 FPGA, with both the vulnerability analysis and experimental setup tailored to this platform. Their glitching method is restricted to toggling between two fixed clock sources with a 0° and 270° phase shift. Hence, their study does not explore the effects of alternative glitch waveforms or phase offsets. While their work demonstrates a misclassification, it lacks a focus on targeted misclassification and a broader exploration of how to fault a specific target class.

Liu et al. demonstrated a clock glitching attack on FPGA-based DNN accelerators using digital signal processing (DSP) units for multiply-accumulate (MAC) operations, requiring up to 10% glitch intensity to achieve high misclassification rates on models like MobileNet and ResNet50 [5]. By contrast, our approach achieves targeted misclassification through a *single-glitch* attack, effectively forcing the model to 'skip' specific classes with significantly *less* effort and *better* success rates.

IV. The Proposed Attack

In this section, we present the threat model, describe the design and deployment of the CNN for FIA experiments, and detail the implementation of our clock glitch attack targeting the CNN's argmax function.

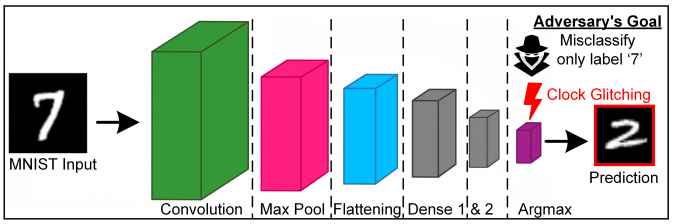

Fig. 1. Targeted clock glitch attack on a seven-layer CNN during AI inference. The CNN processes MNIST digits and classifies them using an argmax unit in the final layer. The adversary injects a clock glitch specifically during the argmax comparison process to misclassify the input class '7'—in this example, as '2'—while leaving other class predictions unaffected. The attacker aims to disrupt only the identification of the digit '7', demonstrating the ability to suppress or redirect specific class predictions during inference.

A. Threat Model

This work targets scenarios where an adversary aims to attack a CNN deployed on an edge device—in our case, an FPGA board. The adversary's goal is to induce misclassifications in a targeted manner, manipulating the CNN's inference process. Unlike prior works that introduce random faults to degrade overall performance [5, 13], our attack is evasive and class-specific. The adversary's focus is on altering only one specific class prediction, without affecting other class predictions. To achieve this, the adversary injects a precisely timed clock glitch during the argmax operation, which alters the correct predicted class. By disrupting the argmax phase, the adversary causes the system to skip the correct class label and select an incorrect one, resulting in a misclassification of that particular class, as shown in Fig. 1.

Our attack model assumes that the adversary has physical access to the device and can thus manipulate the clock signal, similar to the prior well-established paradigms [4, 5, 12, 25]. The attacker's capabilities are limited to a *single-glitch* clock manipulation. To succeed, the adversary must know the exact timing of the argmax operation. We assume that the adversary has prior knowledge of the AI inference implementation and determines the timing of critical operations through cycle-accurate simulations performed before the attack. While such timing information and CNN architecture details could also be obtained via side-channel techniques [33, 34], the extraction process is outside the scope of this paper. Using this knowledge, the attacker can precisely synchronize the glitch with the argmax operation to disrupt the comparison phase. The attack follows a gray-box model, focusing solely on the inference stage, and assumes no access to the model's training data, inference inputs, or weights.

In summary, we follow the common assumption that the attacker (i) has physical access to the hardware, (ii) can inject a *single* clock glitch during the argmax operation of the CNN's inference, (iii) **does not possess knowledge of the model's training data, inference inputs, or weights**, and (iv) determines the timing of the argmax operation through cycle-accurate simulations of the inference implementation. Under these conditions, our proposed attack can reliably force the model to 'skip' the correct class and induce targeted misclassifications, even with limited attacker knowledge and capabilities.

TABLE I
CNN ARCHITECTURE USED FOR MNIST CLASSIFICATION, TRAINED FOR 50 EPOCHS AND ACHIEVING 97.0% ACCURACY. THE MODEL WEIGHTS WERE SUBSEQUENTLY EXPORTED AS QUANTIZED FIXED-POINT FORMAT FOR FPGA-BASED HARDWARE INFERENCE (8-BIT SIGNED).

Layer#	Layer Type	Input Size	Output Size
1	Input Layer	28x28x1	28x28x1
2	Conv2D (Convolution)	28x28x1	26x26x1
3	MaxPooling2D (Pooling)	26x26x1	13x13x1
4	Flattening Layer	13x13x1	169
5	Dense (Fully Connected)	169	64
6	Dense (Fully Connected)	64	10
7	Argmax	10	1

B. Design and Deployment of CNN for FIA Experiments

We implement our attack on the FPGA-based inference implementation of a CNN trained on the MNIST dataset using TensorFlow and Keras in Python. The model is trained offline in floating-point precision (software), while inference is implemented in register-transfer level (RTL) hardware. To address edge device constraints, we implement the CNN using a systolic array architecture with fixed-point quantization and a sequential argmax design, minimizing area overhead, following Google's Coral tensor processing unit (TPU) design philosophy [35]. This design choice makes the implementation suitable for low-cost embedded inference platforms, optimizing both classification accuracy and computational efficiency.

The CNN consists of the following layers:

- **Conv2D layer:** Convolution layer with a kernel size of 3x3, responsible for extracting the input image features.
- **MaxPooling layer:** This layer reduces the spatial dimensions of the feature map, having a kernel size of 2x2.
- **Flatten layer:** Flattens the output from the previous layers for input into the dense layers.
- **Dense layers:** Two fully connected layers, with 64 and 10 neurons respectively, perform a weighted sum of their inputs, apply an activation function (ReLU), and output the result to the next layer.
- **Output layer:** During model training, the softmax layer computes class probabilities, while during hardware inference, the argmax operation selects the predicted class label as an index.

For hardware inference, the output class is determined using a sequential, state-machine-based argmax. This function processes the 10 neuron outputs across multiple clock cycles, comparing each new value to a stored maximum and updating both the maximum value and its index when a larger value is found. Targeted misclassifications are induced by injecting a precise clock glitch during a comparison cycle, causing timing violations that corrupt the registers storing the maximum value and index. The sequential design is particularly well-suited for resource-constrained edge AI devices, as it minimizes hardware usage by reusing a single comparator and a few registers, avoiding the overhead of a large combinatorial tree.

Table I summarizes the overall CNN model's architecture, input, and output sizes used during training and inference, respectively. The model is trained offline (software), whereas inference is performed on the FPGA (hardware). We train the model using floating-point precision and then export the weights in 8-bit fixed-point format. We implement both the convolutional and dense layers using a systolic array architecture inspired by Google Coral TPU's [35]. To enable efficient deployment on FPGAs, we apply fixed-point quantization after each MAC operation. We perform MAC operations in the convolution and dense layers using signed 8-bit fixed-point values for both activations and weights. The CNN model was trained for 50 epochs in floating-point precision using Python, achieving an accuracy of 97.0%. The model was later quantized to fixed-point for FPGA inference, resulting in an accuracy of 94.7%.

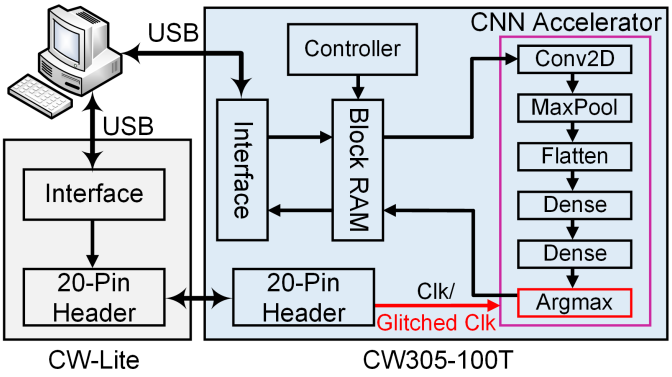

Fig. 2. Overview of the architecture and attack implementation using the ChipWhisper-Lite (attacker) board and the CW305-100T (victim) FPGA board. Under normal operation, a regular clock (Clk) signal is supplied to the victim board. During the attack, the ChipWhisper-Lite injects a single glitched Clk signal, which disrupts the computation. The targeted argmax function is affected by this glitch, resulting in misclassification during the inference process.

C. Clock Glitch Attack Implementation

The attacker focuses on the CNN's output by targeting the argmax operation, which determines the predicted class during inference. This operation is a critical decision point in the hardware, as it registers the final classification. By injecting a precisely timed clock glitch at this stage, the attacker can disrupt the internal register update, causing the predicted class to be skipped and a lower-scoring, incorrect class to be selected. Other class computations remain largely unaffected. This method allows the attacker to induce controlled and repeatable misclassifications rather than random errors, making the attack systematic and highly effective.

Fig. 2 shows the hardware setup used during the attack. The ChipWhisperer-Lite (CW-Lite), acting as the attacker, provides a 100 MHz clock signal to the CW305–100T FPGA board, referred to as the victim device targeted by the attack. To initiate inference, a Python script sends MNIST images to the FPGA board. When the computation reaches the argmax layer, the CW-Lite starts injecting the glitches. The attack campaign systematically sweeps through different glitch width and offset parameters to locate the 'fault window'—the range of parameters that consistently cause misclassifications. By precisely controlling glitch timing, the CW-Lite maps glitch parameters to their effects on the argmax logic, enabling deterministic or highly probable misclassifications rather than random errors. The attack proceeds in two phases: (1) profiling, where effective glitch parameters are identified, and (2) testing, where these parameters are validated for causing targeted misclassifications.

1) Profiling Phase: Identifying Vulnerable Parameters: The profiling phase starts by systematically varying the clock glitch width and offset parameters from -45 to +45[1]. This range is chosen to test how the attack target—running on a clock period of 100MHz (10 nanoseconds)—responds to clock glitches. The CW-Lite generates the clock glitches by shifting the FPGA's internal clock in small steps, which correspond to a percentage of the total clock period. Since the clock period is $10ns$, each clock adjustment (step) represents about 40 picoseconds ($\approx$ 0.4% of the clock period). Thus, the chosen range of offset and width provides the following:

1) **Significant timing deviation.** An effective temporal range of approximately $\pm 1.8ns$ ($45steps \times 40ps/step$ = $1800ps$ = $1.8ns$) relative to the nominal clock edge. This wide sweep ensures that both leading and lagging glitches, as well as glitches extending a substantial portion of the clock cycle, are explored.
2) **Coverage of critical timing windows**. This range directly impacts the setup and hold time margins of sequential logic operating at 100MHz. Glitches within this window are highly likely to induce faults, and exploring such a broad range helps identify the precise glitch parameters that cause anomalous behavior.
3) **Granularity for fine-tuning.** The step size of 1 unit provides a fine granularity of approximately $40ps$, allowing an in-depth mapping of the target's fault injection characteristics and revealing sensitive 'sweet spots'.

To precisely pinpoint the vulnerable time window during which the argmax layer performs its comparisons, images from a single class label (*e.g.,* class '0') are repeatedly sent from the PC to the FPGA. This iterative process allows isolating and compiling a preliminary list of glitch parameters that consistently trigger misclassification for the targeted class while leaving other classes unaffected. The process is subsequently repeated for each of the remaining MNIST digits, from '1' to '9'. This procedure yields a refined list of single-glitch parameters that consistently induce faults for each class. Profiling is time-consuming but only needs to be done once per class and device, making its long-term cost negligible.

2) Testing Phase: Validating Targeted Misclassification: To validate the effectiveness of the shortlisted glitch parameters, randomly selected MNIST images are sent to the FPGA with a designated target class for each image. For example, if the target class is 7, the argmax layer execution is monitored and a precisely timed single glitch is injected using parameters identified during profiling. This glitch is injected exactly when the hardware performs a comparison against target class 7, ensuring that the glitch affects only that specific class comparison, causing it to be skipped and resulting in misclassification. Since the system architecture is known, we profile its behavior to determine the cycles before critical operations, enabling precise glitch injection to misclassify only the target class without affecting others.

[1]The maximum deviation from the nominal (zero) is 45 steps in either direction. Excluding 0, a total of $8,100$ unique *single-glitch* parameter combinations is derived from this range (90 offset $\times$ 90 width = $8,100$).

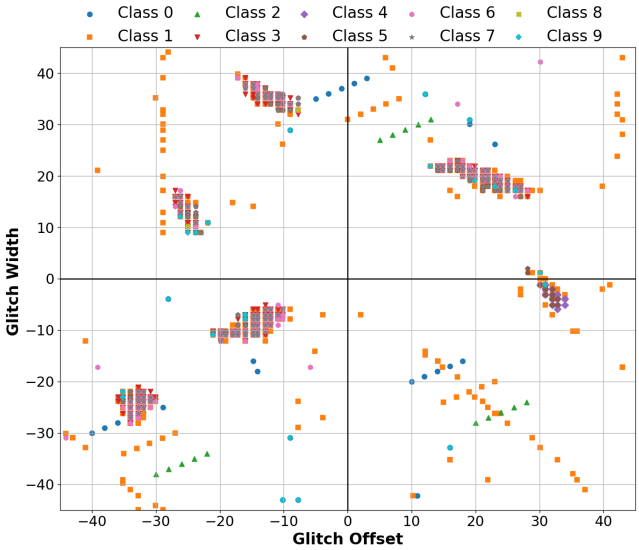

Fig. 3. A comprehensive glitch map illustrating the effective combinations of glitch offset and width that successfully induced faults in the CNN's inference of MNIST digits. Each data point signifies a successful fault injection event, with its color and marker style indicating the 'true' label of the MNIST image that was misclassified. Glitch parameters are presented in clock generator cycles, ranging from -45 to +45 for both glitch width and glitch offset.

TABLE II
CNN MODEL AND TRAINING HYPER-PARAMETERS

Hyperparameter	Value
Input Shape	(28, 28, 1)
Kernel Size (Conv2D)	(3, 3)
Stride (Conv2D)	(1, 1)
Padding (Conv2D)	Valid
Activation (Conv2D)	Rectified linear unit (ReLU)
Number of Kernels	1
Pooling Size (MaxPooling2D)	(2, 2)
Pooling Stride	(2, 2)
Hidden Layer Neurons	64
Output Layer Neurons	10
Activation (Output Layer)	Softmax (training)
Optimizer	Adam
Learning Rate	0.001
Loss Function	Categorical cross-entropy
Epochs	50
Batch Size	64
Validation Split	0.1
Learning Rate	0.001

V. ATTACK RESULTS

To evaluate the proposed attack, we deploy a CNN inference model on the CW305–100T FPGA board and use the CW-Lite platform to perform fault injection. The CNN is trained offline using TensorFlow and Keras with the MNIST dataset [14], utilizing $60,000$ images for training and $10,000$ for testing. The trained model is quantized and implemented at the RTL for real-time inference on the CW305–100T. The CW-Lite is used to inject precisely timed *single* clock glitches during the execution of the argmax layer, enabling a controlled investigation of inference-time misclassifications due to hardware-level faults.

Table II summarizes the training configuration and hyper-parameters. The FPGA area utilization of the CNN design, including look-up tables (LUTs), flip-flops (FFs), block RAMs

TABLE III
TIMING DETAILS AND RESOURCE UTILIZATION FOR CNN ACCELERATOR IN TERMS OF LUTS, FFS, BRAMS, DSP UNITS.

FPGA	LUTs	FFs	BRAMs	DSPs	Freq.	Inference Time
XC7A100T	11079	6880	3	2	100MHz	$0.22ms$

(BRAMs), DSP slices, and latency at a 100 MHz clock frequency are presented in Table III.

Fig. 3 presents the outcome of the profiling phase—a glitch map. During this phase, $8,100$ unique *single-glitch* parameter combinations are tested for each MNIST class by repeatedly sending a single image from that class to the FPGA. This process is repeated for all ten digits (0–9), resulting in a total of $81,000$ experiments. Profiling a single image takes approximately 1.75 hours, but it only needs to be performed once per class and device. Each data point on the glitch map corresponds to a successful fault injection, with color and marker style indicating the true label of the misclassified image. The glitch parameters, expressed in clock generator cycles, span from -45 to $+45$ for both offset and width. The map clearly illustrates the relationship between glitch parameters and misclassifications, highlighting the specific conditions under which the CNN's inference is disrupted the CNN's inference.

Fig. 4 shows the impact of our proposed *single-glitch* attack on CNN inference outcomes. The figure presents experiments involving MNIST images #64 and #41, both with the ground truth label '7'. In the absence of a clock glitch, the model correctly classifies the input as '7', as shown in Fig. 4 (a). However, when the glitch parameters specifically targeting class '7'—identified during the profiling and testing phase—are applied, the model misclassifies the same inputs as '2' and '3', respectively, as depicted in Fig. 4 (b) and (c). These results confirm the efficacy of the attack in reliably inducing targeted misclassifications. Notably, the manipulation causes the model to skip over the intended comparison with class '7', validating the attack's precision in interfering with the comparison logic of the argmax. We only show the results specific to label '7' in the figure to maintain coherence throughout the paper. However, our attack methodology is fully generalizable and capable of targeting and skipping *any targeted* class label.

Table IV summarizes the key parameters and results of our clock glitch attack on an MNIST-trained CNN model, demonstrating a significant decrease in classification accuracy

TABLE IV
KEY PARAMETERS AND OUTCOMES OF A CLOCK GLITCH ATTACK ON AN MNIST-TRAINED CNN MODEL.

Parameters	Details
Inference Platform	FPGA (XC7A100T)
Classification Dataset	MNIST
Training/Testing Images	$60,000/10,000$
Attack Type	Clock glitch
Attack Mode	Single-glitch
# of Glitch Parameters (pre-profiling)	$81,000$
# of Glitch Parameters (post-profiling)	$1,182$
Profiling Time (per class)	≈ 1.75 hours
Model's Accuracy (w/o glitch)	94.7%
Model's Accuracy (w/ glitch)	7.7–14.7%
Attack's Success Rate	80.0–87.0%

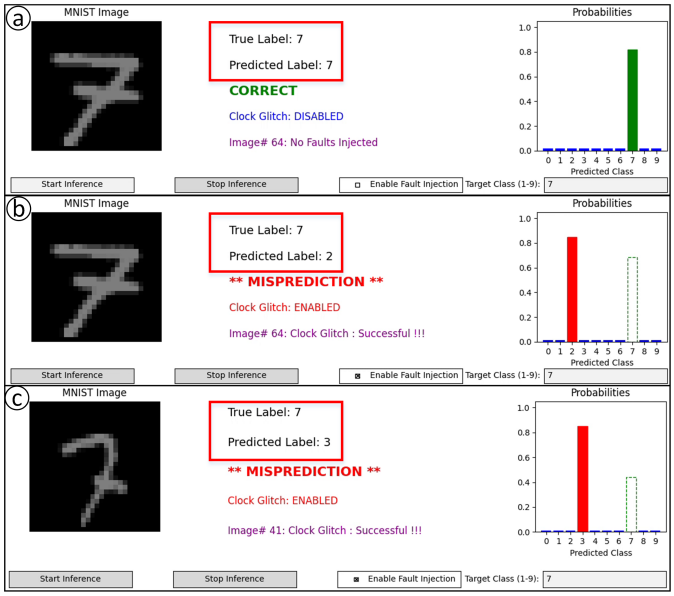

Fig. 4. Inference results for MNIST image #64 and #41 with ground truth label '7'. (a) shows the model's correct prediction without fault injection. (b) and (c) illustrate the impact of the clock glitch attack, where the model, when targeted to misclassify class '7', incorrectly predicts the label as '2' and '3', respectively. This demonstrates the success of the proposed attack in inducing targeted misclassifications and causing 'skip' of a particular class label.

when deployed on an FPGA platform. By applying the *single-glitch* attack to the CNN model on the $10,000$-image MNIST test set, we achieved targeted misclassification rates between 80% and 87%, corresponding to an average attack failure rate of 13–20%. These results demonstrate that the proposed attack on the argmax comparison effectively and reliably causes the model to skip a specific class label.

In our implementation, *successful* attack time ranges from 1 to 10 seconds per image, depending on which glitch combination proves effective for the given *unseen* class image. While this duration exceeds a single inference, it is far more efficient than exhaustively testing all $8,100$ glitch combinations. Our profiling phase reduced the original $10 \times 8,100$ parameter search space to just $1,182$ effective configurations, dramatically reducing the *single-glitch* attack's time complexity. These $1,182$ combinations consistently achieve a misclassification success rate of up to 87%, demonstrating a substantial improvement over the $\approx 10\%$ accuracy achievable through random guessing on the MNIST dataset. **Our targeted attack reduces the CNN's original classification accuracy from 94.7% to an average of 7.7–14.7%**, underscoring its precision and effectiveness in reliably skipping targeted class labels and inducing misclassification on edge devices.

VI. DISCUSSIONS

A. Potential Defenses

The vulnerability of the argmax layer to single clock glitches requires strong defense mechanisms, especially for AI systems in safety-critical applications. Traditional software-based methods are ineffective against hardware-level fault injections, so hardware-centric and design-time countermeasures—such as redundancy, design hardening, clock monitoring, glitch detection, and randomization—can improve security [8, 23, 31]. The optimal strategy depends on the threat model,

performance requirements, and available resources. While a comprehensive evaluation of countermeasures is beyond this work's scope, our study introduces a novel attack vector and highlights a new class of vulnerability in hardware-accelerated DNNs, motivating future dedicated research on defenses.

B. Broader Implications for AI Trustworthiness

The growing use of AI in safety-critical systems makes them vulnerable to targeted hardware faults. We present a novel 'class-skip' attack, which manipulates a neural network to deliberately omit a specific class during classification. Although demonstrated on CNNs using an argmax layer, this approach can be extended to other hardware-implemented decision functions. For example, it could cause particular objects to be misclassified in object detection systems or bypass "unauthorized access" in biometric security. These results highlight the potential risks of such attacks and suggest avenues for future research across a wide range of AI models and applications.

C. Limitations and Future Work

Our work demonstrates a novel form of *targeted misclassification*, which differs fundamentally from prior FIAs that induce only random misclassifications [5]. In particular, the 'class-skip' attack forces a neural network to omit a predetermined class from the classification outcome. For example, the attack can be tailored to prevent the model from ever classifying a digit as '7'. One limitation of this approach is that, while it reliably prevents the target class from being selected, it does not yet control which incorrect class is chosen—so a true '7' might be misclassified as '3' or '9'. Achieving precise, deterministic class redirection would require finer control over the argmax comparison logic, which is a primary focus of our future work.

VII. Acknowledgments

This paper is supported in part by NSF award no. 2333126 and the Office of Naval Research (ONR) grant N00014-23-1-2103. The views, opinions and/or findings expressed are those of the authors and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

VIII. Conclusion

This paper demonstrates the vulnerability of FPGA-accelerated deep learning models to single-glitch attacks. By targeting the argmax layer of a CNN trained on the MNIST dataset, we demonstrate that even subtle clock disruptions can lead to critical misclassifications. We specifically highlight the dangerous outcome of 'skipping' a targeted class label, showcasing a controlled form of adversarial manipulation rather than random error. This poses a significant threat to the trustworthiness and reliability of AI systems in safety-critical applications. Our findings stress the urgent need for robust hardware-level defenses in deep learning systems, particularly on FPGA platforms. Future efforts must prioritize fault-tolerant hardware and a security-by-design approach to ensure the integrity of machine learning systems in critical applications.

References

[1] C. C. Aggarwal *et al.*, *Neural networks and deep learning*. Springer, 2018, vol. 10, no. 978.
[2] A. B. Nassif, I. Shahin, I. Attili, M. Azzeh *et al.*, "Speech recognition using deep neural networks: A systematic review," *IEEE access*, vol. 7, 2019.
[3] S. S. Yadav and S. M. Jadhav, "Deep convolutional neural network based medical image classification for disease diagnosis," *Journal of Big data*, 2019.
[4] B. Yuce, P. Schaumont, and M. Witteman, "Fault attacks on secure embedded software: Threats, design, and evaluation," *Journal of Hardware and Systems Security*, vol. 2, pp. 111–130, 2018.
[5] W. Liu, Chang *et al.*, "Imperceptible misclassification attack on deep learning accelerator by glitch injection," in *2020 Design Automation Conference*, 2020.
[6] F. Aydin, E. Karabulut, and A. Aysu, "Pre-silicon side-channel analysis of ai/ml systems," in *IEEE Physical Assurance and Inspection of Electronics*, 2024.
[7] S. Li, X. Wang, M. Xue, H. Zhu, Z. Zhang, Y. Gao *et al.*, "Yes, One-Bit-Flip matters! universal DNN model inference depletion with runtime code fault injection," in *33rd USENIX Security Symposium*, Aug. 2024, pp. 1315–1330.
[8] J. Mishra and S. K. Sahay, "Modern hardware security: A review of attacks and countermeasures," *arXiv preprint arXiv:2501.04394*, 2025, [cs.CR].
[9] A. A. Malik, H. Mihir, and A. Aysu, "CRAFT: Characterizing and root-causing fault injection threats at pre-silicon," *arXiv preprint arXiv:2503.03877*, 2025.
[10] B. Yuce, N. Ghalaty, H. Santapuri, C. Deshpande, and P. Schaumont, "Software Fault Resistance is Futile: Effective Single-Glitch Attacks," in *Workshop on Fault Diagnosis and Tolerance in Cryptography*, 2016.
[11] Z. Liu, D. Shanmugam, and P. Schaumont, "FaultDetective: Explainable to a Fault, from the Design Layout to the Software," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, no. 4, 2024.
[12] B. Yuce, N. F. Ghalaty, and P. Schaumont, "Improving fault attacks on embedded software using RISC pipeline characterization," in *Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2015, pp. 97–108.
[13] Ordoñez, Jhon and Yang, Chengmo, "Derailed: Arbitrarily Controlling DNN Outputs with Targeted Fault Injection Attacks," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.
[14] F. Chen, N. Chen, H. Mao, and H. Hu, "Assessing four neural networks on handwritten digit recognition dataset (MNIST)," *arXiv:1811.08278*, 2018.
[15] S. Lee *et al.*, "Clock Glitch Fault Attacks on Deep Neural Networks and Their Countermeasures," *Sensors*, vol. 25, no. 9, 2025.
[16] Z. Kazemi, A. Papadimitriou, I. Souvatzoglou, Aerabi *et al.*, "On a low cost fault injection framework for security assessment of cyber-physical systems: Clock glitch attacks," in *2019 IEEE 4th International Verification and Security Workshop (IVSW)*. IEEE, 2019, pp. 7–12.
[17] W. Samek, G. Montavon, S. Lapuschkin, C. J. Anders, and K.-R. Müller, "Explaining deep neural networks and beyond: A review of methods and applications," *Proceedings of the IEEE*, vol. 109, no. 3, pp. 247–278, 2021.
[18] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
[19] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. E. Alsaadi, "A survey of deep neural network architectures and their applications," *Neurocomputing*, vol. 234, pp. 11–26, 2017.
[20] G. Gange and P. J. Stuckey, "The argmax constraint," in *2020 International Conference on Principles and Practice of Constraint Programming*. Springer.
[21] Breier, Jakub and Hou, Xiaolu, "How practical are fault injection attacks, really?" *IEEE Access*, vol. 10, pp. 113 122–113 130, 2022.
[22] A. Barenghi *et al.*, "Fault injection attacks on cryptographic devices: Theory, practice, and countermeasures," *Proceedings of the IEEE*, no. 11, 2012.
[23] Yu, Guangba and Tan, Gou and Huang, Haojia and Zhang, Zhenyu and Chen and others, "A survey on failure analysis and fault injection in AI systems," *ACM Transactions on Software Engineering and Methodology*, 2024.
[24] Z. Kazemi, A. Norollah, A. Kchaou, Fazeli *et al.*, "An in-depth vulnerability analysis of RISC-V micro-architecture against fault injection attack," in *2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 2021, pp. 1–6.
[25] L. Claudepierre, P.-Y. Péneau, D. Hardy, and E. Rohou, "TRAITOR: a low-cost evaluation platform for multifault injection," in *International Symposium on Advanced Security on Software and Systems*, 2021, p. 51.
[26] Jap, Dirmanto and Won, Yoo-Seung and Bhasin, Shivam, "Fault injection attacks on SoftMax function in deep neural networks," in *Proceedings of the 18th ACM International Conference on Computing Frontiers*, 2021.
[27] Breier, Jakub and Hou, Xiaolu and Jap, Dirmanto and Ma, Lei and Bhasin, Shivam and Liu, Yang, "Practical fault attack on deep neural networks," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2204–2206.
[28] Xu, Junge and Zhang, Fan and Jin, Wenguang and Yang, Kun and Wang, Zeke and others, "A Deep Investigation on Stealthy DVFS Fault Injection Attacks at DNN Hardware Accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
[29] J. Richter-Brockmann, P. Sasdrich, and T. Güneysu, "Revisiting fault adversary models–hardware faults in theory and practice," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 572–585, 2022.
[30] P.-Y. Péneau, L. Claudepierre, D. Hardy, and E. Rohou, "NOP-Oriented Programming: Should we Care?" in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2020, pp. 694–703.
[31] S. Tajik and F. Ganji, "Artificial neural networks and fault injection attacks," in *2022 Security and Artificial Intelligence: A Crossdisciplinary Approach*.
[32] F. Khalid *et al.*, "Exploiting Vulnerabilities in Deep Neural Networks: Adversarial and Fault-Injection Attacks," *ArXiv*, vol. abs/2105.03251, 2021.
[33] A. Kurian, A. Dubey, F. Yaman, and A. Aysu, "TPUXtract: An exhaustive hyperparameter extraction framework," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, no. 1, pp. 78–103, 2025.
[34] A. A. Malik, E. Karabulut, A. Awad, and A. Aysu, "Enabling secure and efficient sharing of accelerators in expeditionary systems," *Journal of Hardware and Systems Security*, vol. 8, no. 2, pp. 94–112, 2024.
[35] Y. Sun and A. M. Kist, "Deep Learning on Edge TPUs," *arXiv*, 2108.13732.